```
import os


os.environ["KAGGLE_API_TOKEN"] = "KGAT_b46b600aa2ab67bb270c5735dedb58b6"

print("Kaggle token set ho gaya ")
```

```
Kaggle token set ho gaya
```

```
!pip install -q kaggle
!kaggle datasets list | head
```

```
ref                                                             title
--------------------------------------------------------------  ------------------------------------------------
nalisha/job-salary-prediction-dataset                           Job Salary Prediction Dataset
ssssws/chocolate-sales-dataset-2023-2024                        Chocolate Sales Dataset 2023 - 2024
sumeakash/impact-of-social-media-on-health                      Impact of Social Media on Health
borovai0/student-performance-analytics-dataset                  Student Performance Analytics Dataset
kienngx/nemotron-30b-competition-trainingdata-cot-labels        Nemotron-30B competition trainingdata (CoT+label
asifxzaman/e-commerce-behavior-dataset8000-users                E-Commerce Behavior Dataset(8000 Users)
guriya79/how-ai-is-changing-student-life                        How AI is Changing Student Life
sumeakash/how-ai-is-changing-student-life                       AI Impact on Student Life Dataset
```

```
!kaggle datasets download -d lucario73/radiology-report-diagnosis-findings -p /content/radiology_data
```

```
Dataset URL: https://www.kaggle.com/datasets/lucario73/radiology-report-diagnosis-findings
License(s): ODbL-1.0
Downloading radiology-report-diagnosis-findings.zip to /content/radiology_data
100% 8.24M/8.24M [00:00<00:00, 39.8MB/s]
```

```
import zipfile, os, pandas as pd

zip_path = "/content/radiology_data/radiology-report-diagnosis-findings.zip"
extract_path = "/content/radiology_data/extracted"

os.makedirs(extract_path, exist_ok=True)
with zipfile.ZipFile(zip_path, 'r') as z:
    z.extractall(extract_path)

print("Extracted files:", os.listdir(extract_path))

# Agar sirf ek csv hai:
csv_file = [f for f in os.listdir(extract_path) if f.endswith(".csv")][0]
df = pd.read_csv(os.path.join(extract_path, csv_file))
df.head()
df.columns
```

```
Extracted files: ['radiology_report_PF (1).csv']
Index(['text', 'report'], dtype='object')
```

```
import pandas as pd
import numpy as np

# 1) Basic cleaning
df = df.dropna(subset=["text", "report"]).reset_index(drop=True)

# 2)  columns
df["id"] = df.index
df["report_full"] = df["text"]              # full input
df["doctor_summary"] = df["report"]         # target summary for doctors
df["patient_summary"] = ""                  # yaha baad me model ka output aayega

# 3) Cancer flag ( keyword-based)
cancer_keywords = ["malign", "neoplasm", "carcinoma", "metastat", "tumor", "tumour"]

def has_cancer_terms(text):
    t = str(text).lower()
    return int(any(k in t for k in cancer_keywords))

df["cancer_flag"] = df["doctor_summary"].apply(has_cancer_terms)

# 4) Risk level
df["risk_level"] = np.where(
    df["cancer_flag"] == 1, "High",
    np.where(df["report_full"].str.len() > 300, "Medium", "Low")
)

# 5) Length features
```

```
df["report_length"] = df["report_full"].str.len()
df["summary_length"] = df["doctor_summary"].str.len()

# 6) text features
df["avg_word_length"] = df["report_full"].apply(
    lambda x: np.mean([len(w) for w in str(x).split()]) if len(str(x).split()) > 0 else 0
)
df["num_numbers"] = df["report_full"].str.count(r"\d")

print("Final shape:", df.shape)
print(df.columns)
df.head()
# 25k ka subset for Colab GPU
small_df = df.sample(n=8000, random_state=42).reset_index(drop=True)
```

```
Final shape: (89395, 12)
Index(['text', 'report', 'id', 'report_full', 'doctor_summary',
       'patient_summary', 'cancer_flag', 'risk_level', 'report_length',
       'summary_length', 'avg_word_length', 'num_numbers'],
      dtype='object')
```

```
df.to_csv("/content/radiology_enhanced_full.csv", index=False)
print("Saved radiology_enhanced_full.csv ")
```

```
Saved radiology_enhanced_full.csv
```

```
from sklearn.model_selection import train_test_split

train_df, test_df = train_test_split(
    small_df, test_size=0.2, random_state=42
)
print(len(train_df), len(test_df))
```

```
6400 1600
```

```
!pip install -q transformers datasets sentencepiece rouge-score

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using:", device)
```

```
Using: cuda
```

```
model_name = "t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

# Ab model ko device pe bhejo
model = model.to(device)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/t
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json:        1.21k/? [00:00<00:00, 34.8kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limi
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
tokenizer_config.json:        2.32k/? [00:00<00:00, 67.7kB/s]
spiece.model: 100%                                                    792k/792k [00:00<00:00, 5.44MB/s]
tokenizer.json:        1.39M/? [00:00<00:00, 6.99MB/s]
model.safetensors: 100%                                               242M/242M [00:02<00:00, 237MB/s]
Loading weights: 100%                                                 131/131 [00:00<00:00, 259.48it/s, Materializing param=shared.weight]
generation_config.json: 100%                                          147/147 [00:00<00:00, 7.47kB/s]
```

```
from torch.utils.data import Dataset, DataLoader

class RadReportDataset(Dataset):
    def __init__(self, df, tokenizer, max_input_len=128, max_target_len=64):
```

```
        self.df = df.reset_index(drop=True)
        self.tok = tokenizer
        self.max_input_len = max_input_len
        self.max_target_len = max_target_len

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        source = "summarize: " + row["report_full"]
        target = row["doctor_summary"]

        x = self.tok(
            source, max_length=self.max_input_len,
            truncation=True, padding="max_length", return_tensors="pt"
        )
        y = self.tok(
            target, max_length=self.max_target_len,
            truncation=True, padding="max_length", return_tensors="pt"
        )

        labels = y["input_ids"].squeeze()
        labels[labels == self.tok.pad_token_id] = -100  # ignore pad in loss

        return {
            "input_ids": x["input_ids"].squeeze(),
            "attention_mask": x["attention_mask"].squeeze(),
            "labels": labels,
        }

train_ds = RadReportDataset(train_df, tokenizer, max_input_len=256, max_target_len=64)
test_ds  = RadReportDataset(test_df, tokenizer, max_input_len=256, max_target_len=64)

train_loader = DataLoader(train_ds, batch_size=1, shuffle=True)
val_loader   = DataLoader(test_ds, batch_size=1)
```

```
from torch.optim import AdamW
from tqdm.auto import tqdm

optimizer = AdamW(model.parameters(), lr=2e-5)

def train_epoch():
    model.train()
    total = 0
    for batch in tqdm(train_loader):
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        optimizer.zero_grad()
        torch.cuda.empty_cache()
        total += loss.item()
    return total / len(train_loader)

def eval_epoch():
    model.eval()
    total = 0
    with torch.no_grad():
        for batch in tqdm(val_loader):
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            total += outputs.loss.item()
    return total / len(val_loader)

for epoch in range(1):
    train_loss = train_epoch()
    val_loss   = eval_epoch()
    print(f"Epoch {epoch+1}: train_loss={train_loss:.4f}, val_loss={val_loss:.4f}")

    # save after each epoch
    torch.save(model.state_dict(), f"/content/t5_epoch{epoch+1}.pt")
```

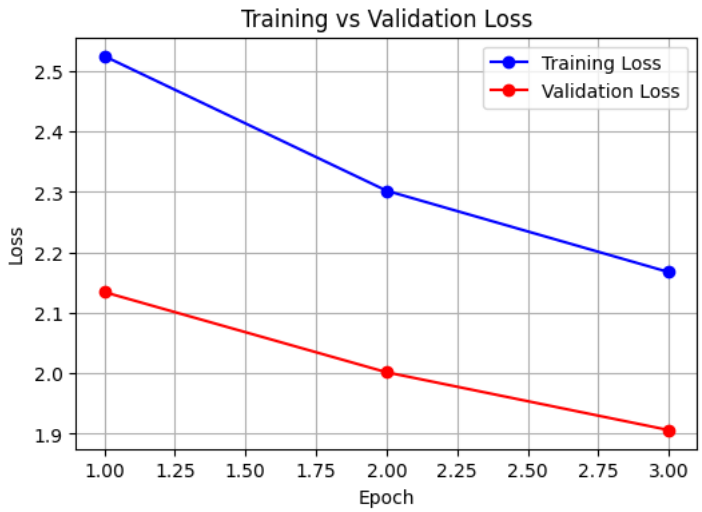
```
100%                                          6400/6400 [12:05<00:00,  8.11it/s]

100%                                          1600/1600 [00:33<00:00, 50.51it/s]
Epoch 1: train_loss=3.2035, val_loss=2.4057
```

```
train_losses = []
val_losses = []

for epoch in range(3):   # ya jitne bhi epochs kar rahi ho
    train_loss = train_epoch()
    val_loss   = eval_epoch()

    train_losses.append(train_loss)
    val_losses.append(val_loss)

    print(f"Epoch {epoch+1}: train_loss={train_loss:.4f}, val_loss={val_loss:.4f}")
```

```
100%                                      6400/6400 [11:23<00:00,  9.57it/s]

100%                                      1600/1600 [00:34<00:00, 36.32it/s]

Epoch 1: train_loss=2.5249, val_loss=2.1345
100%                                      6400/6400 [11:20<00:00,  9.76it/s]

100%                                      1600/1600 [00:33<00:00, 47.58it/s]

Epoch 2: train_loss=2.3022, val_loss=2.0017
100%                                      6400/6400 [11:20<00:00,  7.92it/s]

100%                                      1600/1600 [00:34<00:00, 48.06it/s]

Epoch 3: train_loss=2.1674, val_loss=1.9060
```

```
!pip install -q rouge-score

from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(
    ['rouge1', 'rouge2', 'rougeL'], use_stemmer=True
)

def generate_summaries(df_sample, max_input_len=128, max_target_len=64):
    model.eval()
    preds, refs = [], []
    for _, row in df_sample.iterrows():
        src = "summarize: " + row["report_full"]
        inputs = tokenizer(
            src, max_length=max_input_len,
            truncation=True, return_tensors="pt"
        ).to(device)
        with torch.no_grad():
            out_ids = model.generate(
                **inputs,
                max_length=max_target_len,
                num_beams=4,
                early_stopping=True
            )
        pred = tokenizer.decode(out_ids[0], skip_special_tokens=True)
        preds.append(pred)
        refs.append(row["doctor_summary"])
    return preds, refs

# test set ke 30 examples pe ROUGE:
sample_df = test_df.sample(n=30, random_state=42)
preds, refs = generate_summaries(sample_df)

agg_scores = {"rouge1": [], "rouge2": [], "rougeL": []}
for ref, pred in zip(refs, preds):
    s = scorer.score(ref, pred)
    for k in agg_scores:
        agg_scores[k].append(s[k].fmeasure)

for k in agg_scores:
    print(k, sum(agg_scores[k]) / len(agg_scores[k]))
```

```
rouge1 0.2820610637201827
rouge2 0.08036215044305457
rougeL 0.20489218574772664
```

```
import matplotlib.pyplot as plt

epochs = range(1, len(train_losses) + 1)

plt.figure(figsize=(6,4))
plt.plot(epochs, train_losses, 'b-o', label='Training Loss')
plt.plot(epochs, val_losses, 'r-o', label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
```

```
plt.title('Training vs Validation Loss')
plt.legend()
plt.grid(True)
plt.show()
```